

CO1 - AT - 1 - Short Answer Analysis

- 1) Suppose a modern Operating system supports multiple users and processes with shared resources like CPU, memory and I/O devices.
- (a) classify diff't types of O.S suitable for this Scenario + justify your choice.
 - b) Draw the structure of an O.S showing how resources are managed.
 - (c) write a sequence of system calls required to create + execute a new process in such an environment.
- 2) Consider an Operating system managing multiple files and devices for concurrent users.
- (a) Categorize system calls used for file + device management.
 - (b) Illustrate how the OS handles file operations using a layered structure diagram.
 - (c) write a pseudo-cdb or C-based system call sequence to open, read + close a file.
- 3) Assume a real-time Operating system used in an embedded system controlling industrial machinery.
- (a) identify the type of OS + explain why it is suitable for real-time applications.
 - (b) Draw a diagram representing the interaction b/w hardware, kernel + user processes.

(c) List + explain system calls required for process synchronization in this system.

4) A distributed computing system consists of multiple interconnected computers sharing resources over a network.

a) Classify the type of O.S used and compare it with centralized OS.

b) Draw the architecture of distributed OS showing communication b/w nodes.

c) Write the sequence of system calls involved in Interprocess communication.

5) Consider a scenario where an Operating system must manage process scheduling, memory allocation + device communication efficiently.

(a) Explain the major functions of an OS in handling these tasks.

(b) Draw a block diagram representing OS components and their interactions.

(c) Write system call examples for memory allocation + device management.

1)

a) Diff. Types of Operating Systems:

1) Multuser Operating System:-

Allows multiple users to access the system simultaneously.

2) Multitasking Operating system - Executes multiple tasks at same time.

3) Multiprocessing operating system:-

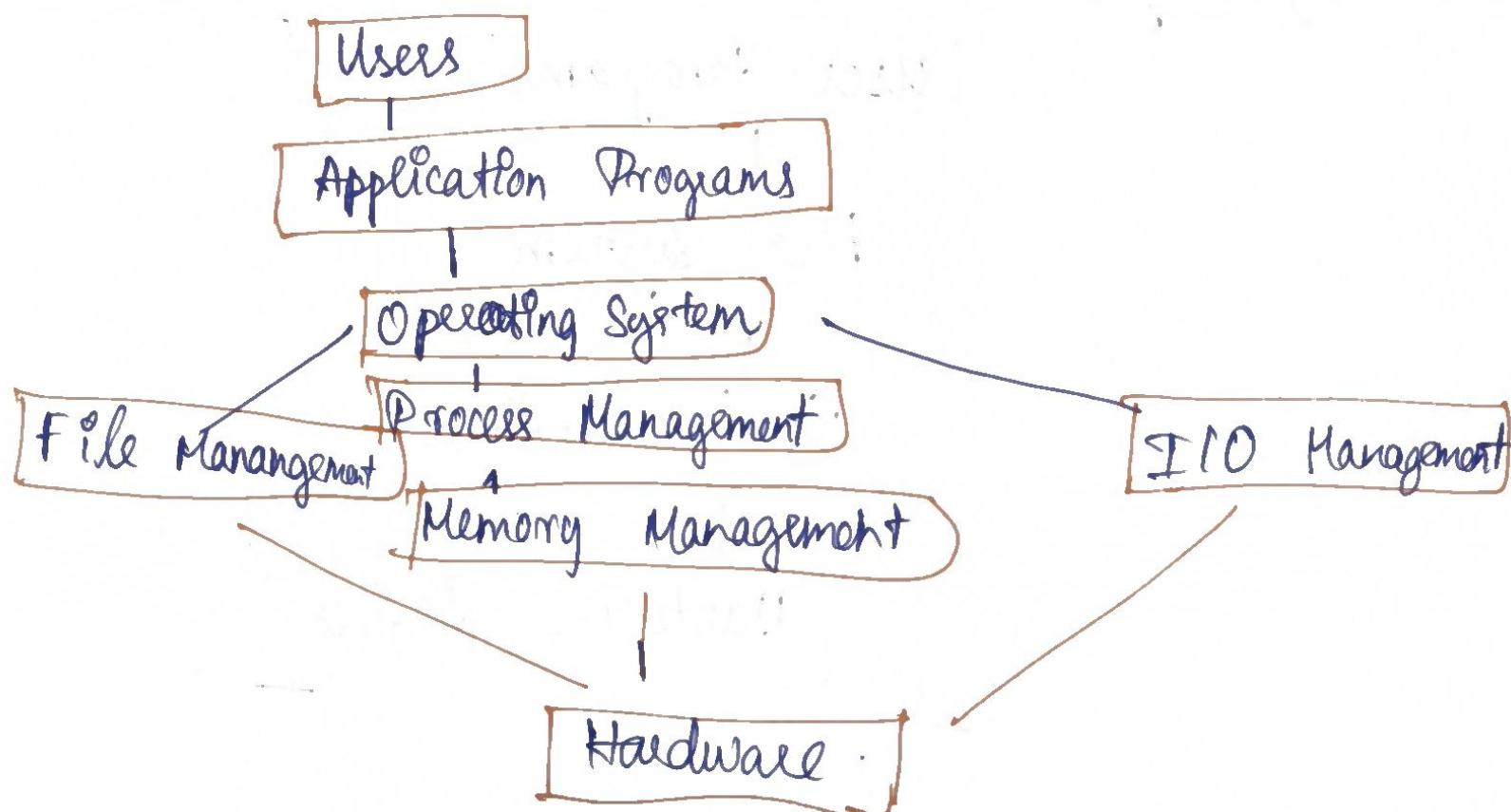
Uses multiple CPUs to improve performance.

4) Time sharing operating system.

Shares CPU Time among users.

Justification: These OS types efficiently share CPU, Memory & I/O resources among many users & processes.

b)



(c) System Call Sequence :

```
fork(); // Create child process  
exec(); // Load & Execute new program  
wait(); // Parent waits for child  
exit(); // Terminate process.
```

2)

(a) File & Device Management System Calls.

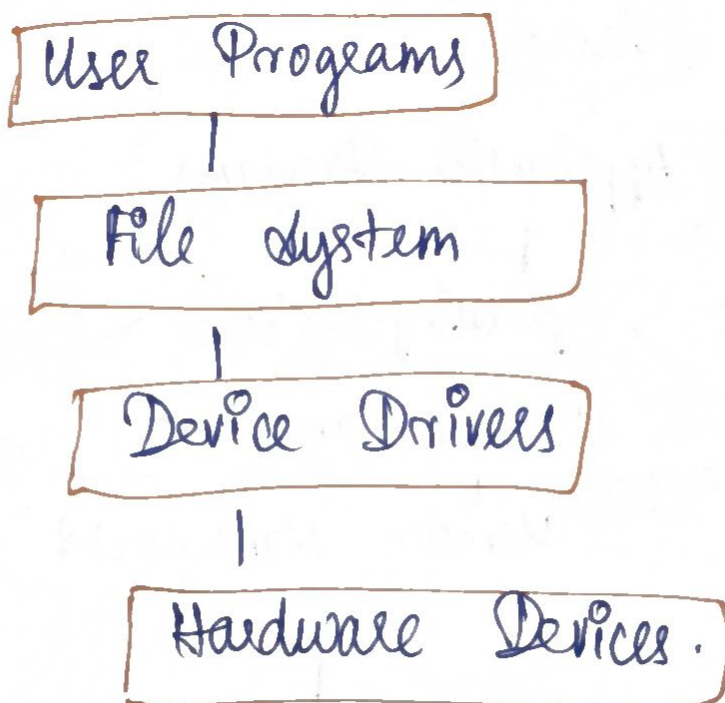
File Management

- ⇒ Create()
- ⇒ Open()
- ⇒ Read()
- ⇒ Write()
- ⇒ Close()
- ⇒ Delete()

Device Management.

- ⇒ Open()
- ⇒ Read()
- ⇒ Write()
- ⇒ ioctl()
- ⇒ Close()

b) Layered Structure:



c) C system Call sequence:

```
int fd;  
char buffer[100];  
fd = open("file.txt", O_RDONLY);  
read(fd, buffer, 100);  
close(fd);
```

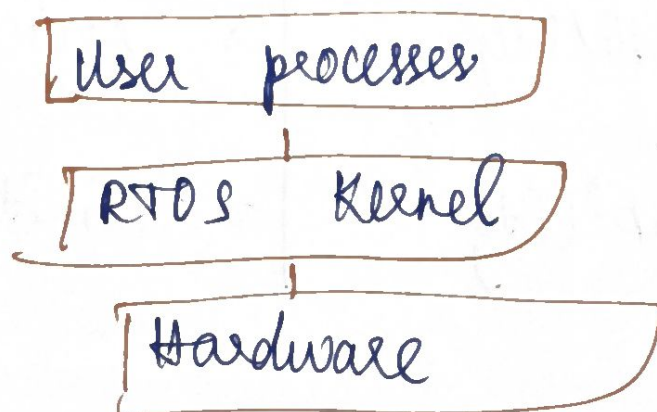
3) a) Type of OS:

Real-time Operating System (RTOS).

Reason:

- ⇒ Fast Response Time.
- ⇒ Predictable execution.
- ⇒ Meets strict deadlines.
- ⇒ Suitable for industrial automation & embedded systems.

b) RTOS structure:



(c) Process Synchronization System Calls:

`fork()` - Creates Process

`wait()` - Waits for process completion

`sem_wait()` - Locks semaphore

`sem_post()` - Releases semaphore

`pthread_mutex_lock()` - Locks mutex

`pthread_mutex_unlock()` - Unlocks mutex.

4)

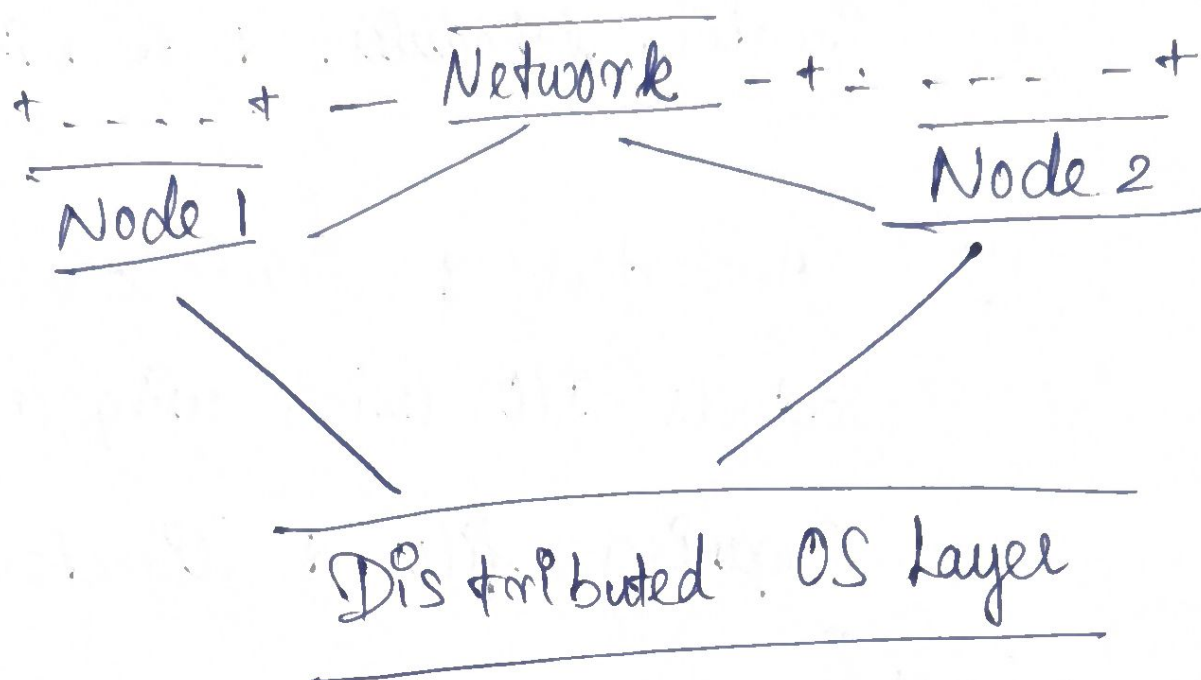
a) Type of OS:

The type of OS used is Distributed operating system.

Comparison:

Distributed OS	Centralized OS
⇒ Multiple Computers	⇒ Single Computer
⇒ Resource Sharing	⇒ Limited sharing
⇒ High Reliability	⇒ Lower Reliability
⇒ Better Scalability	⇒ Limited Scalability.

b) Distributed OS Architecture :



c) IPC system Calls:

⇒ pipe() - Communication channel b/w two related processes.

⇒ fork() - Creates a new child process from the Parent process

⇒ Read() - Reads data from a file, pipe

⇒ Write() - Writes data from a file

⇒ close() - Closes an open file

⇒ msgget() - Creates or accesses a message

⇒ msgsnd() - Sends a message

⇒ msgrcv() - Receives a message

⇒ shmget() - Creates or accesses a shared memory

⇒ shmat() - Attaches a shared memory.

5)

a) Major functions of O.S:

Process Management - Creates, schedules, + terminates processes.

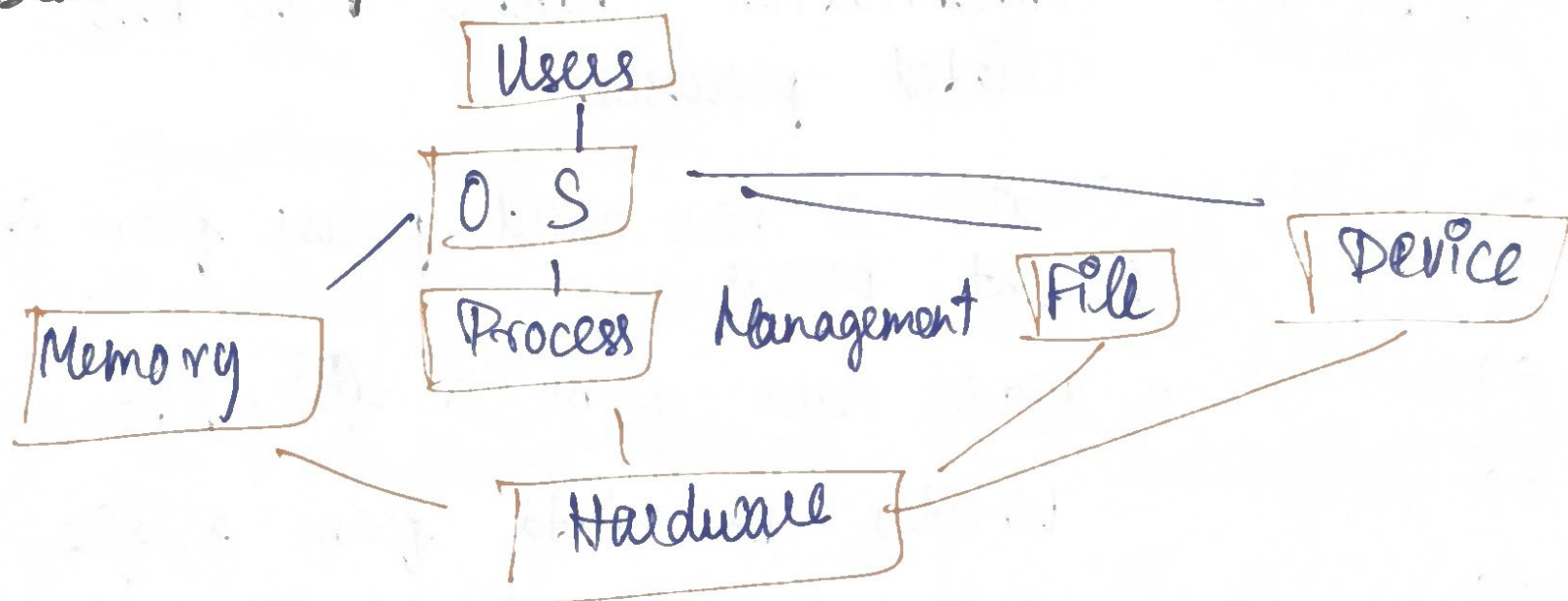
Memory Management - Allocates + deallocates memory.

Device Management: Controls I/O devices using drivers

File Management: Organizes files + directories

Security: Protects users + system resources.

b) Block Diagram:



c)

System Calls

Memory Management

Malloc();

free();

mmap();

munmap();

Device Management.

open();

read();

write();

ioctl();

close();